

Performance evaluation for the hash generation phase of a democratic blockchain

Luis Lugo* and Cesar Pedraza

Department of Systems and Industrial Engineering,
Universidad Nacional de Colombia,

Bogota, DC, Colombia

Email: lelugom@unal.edu.co

Email: capedrazab@unal.edu.co

*Corresponding author

Abstract: Distributed ledger technologies (DLTs) have the potential to transform different areas of research and industry. Initially created to support a peer-to-peer electronic cash system – more commonly known as bitcoin – blockchains provide a decentralised transactions and data management technology. This decentralised technology is secure, anonymous, and transparent. Nonetheless, the blockchain protocol has a number of technical weaknesses such as power computing dependency and high power consumption. An alternative protocol proposes a democratic approach in which the computing power is not a determinant factor in user participation. The democratic protocol has three phases: hash generation, hash broadcast, and hash validation. We evaluate the performance of the hash generation phase on CPU, GPU, and cluster platforms. A considerable speedup is achieved with GPUs when using final nonce values similar to a real blockchain application. Also, it provides the best power efficiency.

Keywords: distributed ledger technology; DLT; hash generation; parallel computing.

Reference to this paper should be made as follows: Lugo, L. and Pedraza, C. (2020) 'Performance evaluation for the hash generation phase of a democratic blockchain', *Int. J. Internet Technology and Secured Transactions*, Vol. 10, No. 3, pp.286–303.

Biographical notes: Luis Lugo received his BS in Electronics Engineering from the National University of Colombia, Bogotá, Colombia in 2009. He is currently pursuing his MS degree in the Systems and Industrial Engineering Department, National University of Colombia, Bogotá, Colombia. From 2009 to 2016, he worked at the engineering operations in the energy sector. His research interests include distributed ledger technologies, recurrent neural networks and bioinformatics.

Cesar Pedraza received his BS in Electronics Engineering from the Santo Tomás University, Bogotá, Colombia, in 2001, MS in Electronics Engineering from the Los Andes University, Bogotá, Colombia, in 2005 and his PhD in Informatics from the Rey Juan Carlos University, Madrid, Spain in 2010. He is currently a Assistant Professor in the Department of Systems and Industrial Engineering, National University of Colombia. His main research interests are IoT devices development, intelligent transportation systems, digital signal processing, neural networks and VLSI system design.

1 Introduction

The blockchain appeared as the distributed ledger technology (DLT) behind Bitcoin (Nakamoto, 2008), a peer-to-peer electronic cash system. Although initially designed for peer-to-peer payment systems, the blockchain has found applications in diverse areas of research and industry. Among those use cases, we can find applications in supply chains, property licensing, electronic voting, digital identity, distribution of electricity for electric cars, smart contracts and healthcare systems (The Current State of the Healthcare Blockchain, 2017; Yli-Huumo et al., 2016). The decentralised technology provides security, anonymity, privacy and transparency (Yli-Huumo et al., 2016; Pilkington et al., 2017). It also guarantees data integrity through the use of cryptographic techniques.

The transfer of value in the blockchain network is known as a transaction. The system identifies the users involved in a transaction with an address, which is the public key in a public-private key pair. Transactions are assembled in Merkle trees to verify all the contents. These transactions are stored in blocks. All the blocks form a chain because each block has a hash value pointing to the previous block. The chain of blocks – more commonly known as blockchain – forms a public ledger of transactions that are replicated on all the nodes connected to the system. Users verify blocks by computing their hash values, thus, any malicious change or double spending transaction will be identified because of the change in hash values (Paul et al., 2014).

Original blockchain security is based on the proof-of-work (PoW) principle, which establishes a limit of 51% in system processing capacity for any node (Nakamoto, 2008). This raises the possibility that a node surpasses such capacity in order to configure an attack to the system. Moreover, blockchain technology has a number of weaknesses from the technical protocol and the financial incentive in block generation (Yli-Huumo et al., 2016; Paul et al., 2014): unfair competition among miners because rich get richer and poor get poorer, block races and selfish mining, illegal use of machines for mining, wastage of computer power and energy – USD 15M/day in wasted energy, no guarantee of coin generation at a fixed rate, slow speed of transactions at 7 tps (transactions per second), security, usability, versioning, hard forks and multiple chains.

Paul et al. (2014) propose a democratic blockchain protocol that guarantees the participation of every node in the network, without benefiting users with large computing power. Limitations of the approach include a higher message complexity than the original Proof-of-Work protocol in bitcoin because every node exchanges messages. By the same token, there is a flooding of messages in the network that supports the democratic blockchain protocol.

The increasing complexity of block generation in bitcoin has enabled the creation of parallelising strategies to boost computing performance. Dev (2014) implements a system for block generation using the simultaneous execution of CPU and GPU computing power. He evaluates the performance using three configurations: CPU only, GPU only and CPU + GPU, getting higher performance for the combined setup.

On the other hand, companies such as Butterfly Labs¹ created ASICs and custom devices for block generation. CGminer² software implements the block generation phase using FPGA and ASIC devices. BFGMiner³ software integrates CPUs, GPUs, FPGAs and ASICs in a modular architecture to improve parallel computing performance for block generation.

In this paper we parallelise the hash generation phase in Paul et al.'s (2014) democratic blockchain protocol, conducting the update of the block format to account for the already published SHA-1 practical collision (Stevens et al., 2017) and evaluating a number of parallel programming approaches to improve computational efficiency. Parallel programming approaches under consideration include CPU (POSIX, OpenMP), GPU (CUDA and OpenCL) and cluster (Open MPI).

This paper is organised as follows. Section 2 revises the hash generation phase and hashing function details. Parallel implementation and techniques for performance evaluation appear in Section 3. Section 4 presents experimental results for the parallel techniques under consideration. Finally, Section 5 states the conclusions.

2 Hash generation phase

Paul et al. (2014) propose a fair scheme for generating blocks in the system. Instead of awarding the node that finds the first block hash lower than a target value, the proposed scheme awards the node that finds the minimum hash value in a specific time span. The new scheme guarantees security by the timestamp in each block. It also creates a fair protocol for block generation, enables the creation of blocks at a fixed rate and decreases the amount of energy spent in block generation. The scheme is based on three phases: hash generation, hash broadcast and hash validation.

A hash function $H: \{0, 1\}^* \rightarrow \{0, 1\}^n$ is an iterative algorithm that takes a message of arbitrary length and generates a fixed length output with n bits, known as a message digest or hash value. $n = 256$ for the SHA256 protocol and $n = 160$ for the SHA-1 protocol. Both SHA256 and SHA-1 pad the input message with 0 to get a message length that is a multiple of 512 bits. Then, through transformation rounds – also known as operations – the algorithm calculates the hash value (Stevens et al., 2017; Michail et al., 2016).

Although there are a finite number of possible outputs for an infinite number of inputs, the hash value is considered unique because it is computationally infeasible to find two different messages that generate the same hash value (Michail et al., 2016). Therefore, the strength of a hashing function relies on the fact that calculating the hash is computationally feasible, while finding the message that originated the hash is computationally too expensive for any brute force search or practical implementation. When a technique for practical collisions appears, the hashing function is no longer a safe alternative. A collision happens when given two different messages x ; y , the hash function results are identical $H(x) = H(y)$ (Stevens et al., 2017).

This kind of function is quite common for digital signature schemes, message authentication codes, password hashing and content-addressable storage (Stevens et al., 2017; Michail et al., 2016).

The SHA-1 protocol was created back in 1995 and have been widely used in various applications, including the GIT versioning system, document signatures and TLS certificate signatures. Although multiple analyses and theoretical attacks made the SHA-1 protocol deprecated in 2011, it continued to be used because no practical collision existed (Stevens et al., 2017). Stevens et al. (2017) published the first known instance of a

practical collision for the SHA-1 protocol in 2017. Although the collision is computationally expensive, the technique is around 100,000 times faster than a brute force attack. Thus, existing applications using SHA-1 protocol should update to a more robust hashing protocol.

Table 1 Block header fields for generating the hash in the blockchain

<i>Field</i>	<i>Bytewidth</i>	<i>Description</i>
Version	4	Header version
Previous hash	32	Digest of the previous block header
Merkle root	32	Root of transactions validated in the header
Time	4	Timestamp in UNIX format
User address	32	Digest of the receiving user public key
Nonce	4	ser adjusted value

Source: It has a slight change in the address field from the header proposed by Paul et al. (2014)

3 Performance evaluation

The hash generation phase of the democratic blockchain was tested in different parallel platforms. CPU, GPU and cluster platforms were examined to determine the performance of a parallel version of the hash generation.

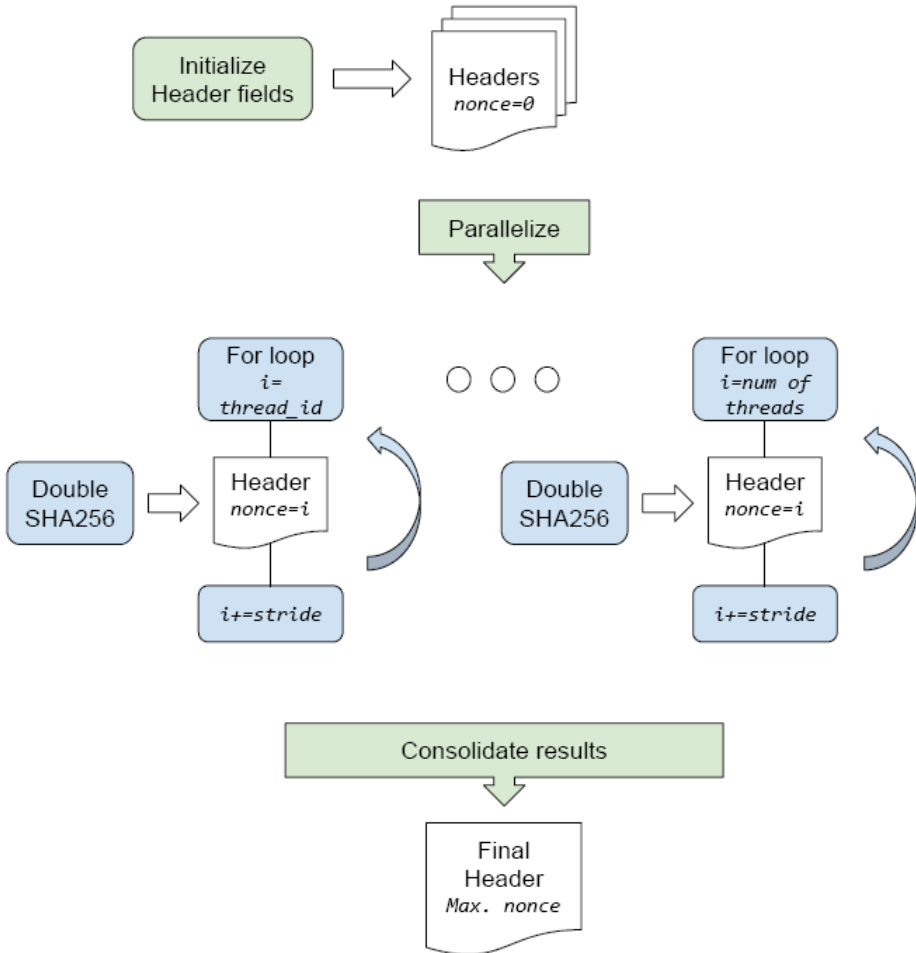
We performed a slight change in the block header proposed by Paul et al. (2014): instead of using 20 bytes for the user address field, we increased the field to 32 bytes. The increase of the address field length allows the use of a stronger algorithm for the user address hash. Thus, we no longer use the SHA-1 encoded user public key but its SHA256 hash (Table 1).

The SHA256 function is at the heart of the blockchain protocol. The block hash generation has an essential role in the construction of the blockchain because the hash is a unique identifier for each block. It is also the cryptographical mechanism used to validate the content and the transactions stored in each block (Nakamoto, 2008). To perform the hash generation phase, we use the OpenSSL SHA256 implementation for calculating the double hash of the block.

The system partitions the hash generation phase using a cyclic decomposition approach (Barney, 2017), where each thread executes each own set of iterations from the main loop (Figure 1). To avoid inconsistent results when computing the hashes and possible delays that could be generated when multiple threads try to access the exact same memory position, we create a block for every thread in the cyclic decomposition, copying constant values in the block fields before the loop starts.

Existing alternatives for implementing multi-threading mechanisms in software applications include POSIX, OpenMP, Open MPI, CUDA and OpenCL. They allow the implementation of parallel applications, which solve computational problems using multiple compute resources simultaneously (Barney, 2017).

Figure 1 Cyclic decomposition strategy to improve the computing performance of the hash generation phase (see online version for colours)



3.1 CPU

Nowadays CPU architectures include multiple processing cores to run parallel programs efficiently. These architectures are very attractive to run parallel hash calculations in blockchain applications. There are different paradigms to program parallel programs for CPUs. One of the most standard ways to run threads is POSIX. The portable operating system interface (POSIX) provides a set of standard abstractions to provide portability on UNIX operating systems. It has been around from the 1980s, when fragmentation in UNIX systems was a serious concern. POSIX is an IEEE standard that comprises specifications for OS aspects such as directory structure, command-line utilities, environment variables and functions at the system level. It also has implementations in the C programming language for OS abstractions. Such abstractions include inter-process

communication (IPC), signals, streams, realtime, threads and sockets (Atlidakis et al., 2016; Taha et al., 2017).

POSIX provides the Pthread library for supporting threads in applications. Threads constitute a lightweight mechanism for parallel or concurrent execution of code. They are set of instructions that can be independently or simultaneously scheduled by the OS. They run inside a UNIX process with an independent flow of control. Although threads maintain their own copy of essential resources, they share most of the process resources while the process exists in the OS (Figure 2). Thus, memory access should be synchronised to avoid software malfunctions (Barney, 2017).

OpenMP⁴ is an application programming interface (API) for shared memory parallel programs. It supports C, C++ and Fortran programming languages. The specification is supported by major software and hardware vendors. Moreover, the specification is maintained by the OpenMP architecture review board (ARB) (Barney, 2017; Pas, 2009).

The API comprises three main components: compiler directives, runtime library routines and environment variables. Its main characteristics include (Barney, 2017; Pas, 2009):

- good performance
- standardisation
- scalability
- portability across a large number of compilers
- Unix/Linux and Windows support
- ease of use, with little programming effort
- support for incremental parallelisation of programs.

High-level APIs such as OpenMP uses the standard POSIX for implementing the functionality. Indeed, although software applications use high-level libraries and frameworks nowadays, a lot of those libraries and frameworks rely on POSIX standards to provide portability across different OS's. Also, high-level libraries and frameworks could implement some functionality using OS-specific interfaces (Atlidakis et al., 2016).

3.2 GPU

Graphics processing units (GPUs) are very powerful architectures to run parallel programs. These devices have evolved to the point of having thousands of cores in a single chip with a very low power consumption per core. There are different ways to program applications on GPU devices. Nowadays, CUDA and OpenCL are two of the most important architectures to develop parallel programs.

3.2.1 CUDA

The compute unified device architecture (CUDA) is a parallel computing platform and a programming model provided by NVidia (Harris, 2017a). It supports programming using C, C++ and Fortran. To compile the programs accessing the GPU, you need to install the CUDA toolkit, which is freely available from NVidia. The CUDA toolkit⁵ is a

development environment to develop, optimise and deploy applications to GPU enabled computing platforms. It also has libraries for linear algebra, image and video processing, deep learning and graph analytics. Moreover, the toolkit is available for Windows, Linux and Mac OSX.

GPUs with the Pascal architecture implement a unified memory model to ease the software development process and improve program performance (Sakharnykh, 2016). The model enables the sharing of CPU and GPU memory through a single virtual memory space. It also allows automatic page migration through the Page Migration Engine, which provides hardware support for virtual memory page faulting and migration. It is important to make sure initialised variables are available in the GPU memory before kernel execution. The function *cudaMemPrefetchAsync* is a practical alternative and should be called before kernel invocation. Another option is to create a kernel for initialising variables before calling the kernel for computations (Harris, 2017b).

Back in 2010, Apple Inc. proposed the open computing language (OpenCL) and open, royalty-free and cross-platform standard that supports a number of computing platforms, including servers, personal computers, mobile devices and embedded systems (Liebling and Khanna, 2017; The Open Standard for Parallel Programming of Heterogeneous Systems, 2017). It is a platform and vendor agnostic library for parallel computations, providing portability across computing platforms and easing the process of developing high-performance computing applications (Liebling and Khanna, 2017).

In the OpenCL parallelising approach, the CPU is the host. It uses the OpenCL API to initialise, compile, submit, queue and synchronise computing resources. Accelerating devices such as GPUs, Processors, DSPs, etc. act as coprocessors, with their own memory. The device executes its computations based on a kernel implementation that uses the C language. In runtime, the device employs groups of threads known as work-groups. Such groups can have up to three dimensions and are executed in parallel by the device cores (Liebling and Khanna, 2017).

As it is not possible to use OpenSSL functions from a CUDA or OpenCL-GPU kernel, a custom implementation of the SHA256 algorithm was performed based on the description at Descriptions of SHA-256, SHA-384 and SHA-512 (2017). It supports message lengths of up to four 512-bit blocks, which is enough for our 108-bytes header. The implementation is tested with the examples provided in the reference (Descriptions of SHA-256, SHA-384 and SHA-512, 2017). The padded message register in the SHA256 implementation is declared in a GPU function. The implementation copies the header from the global memory to the local register. It also declares locally all the variables needed for the hash computation. Because of this, we take advantage of the register memory in each thread, which is faster than the GPU global memory.

Regarding the GPUCUDA implementation, the *cudaMemPrefetchAsync* is called before the kernel, to prefetch block header array data to GPU global memory before starting the computation. Both in CPU and GPU parallelisation, every thread stores the computed double hash in its own block as the nonce increases. When all threads are done, we take the header matching the highest value of nonce, which is taken from the example block header (Table 2).

To perform the implementation using OpenCL, we use the same strategy as the CUDA implementation, converting *blocks * threads* into 2-dimensional work-groups. The first work-group dimension is equivalent to the number of blocks while the second one is the number of threads. By varying the first dimension of the work-groups, the

performance is evaluated for further comparison to the CPU times. Moreover, the second dimension is set to a fixed value of 128, the same thread count used with CUDA. Both the kernel and the SHA256 implementations are stored in the same file for subsequent compilation by OCL⁶. Local variables are created inside the kernel to store header, hash and result nonce values. When the work-group is done, local results are stored back in global memory.

The starting value for the iteration and the stride for the parallelisation are calculated using:

$$i_0 = \text{thread_id}$$

$$\text{stride} = \text{number_of_threads}$$

The Open MPI implementation calculates the starting value and the stride as follows:

$$i_0 = \text{number_of_threads} * \text{instance_id} + \text{thread_id}$$

$$\text{stride} = \text{number_of_instances} * \text{number_of_threads}$$

On the other hand, the starting value and the stride for the GPU parallelisation are calculated using:

$$i_0 = \text{blockIdx.x} * \text{blockDim.x} + \text{threadIdx.x}$$

$$\text{stride} = \text{blockDim.x} * \text{gridDim.x}$$

where *blockIdx.x* is the block identifier, *threadIdx.x* is the thread identifier, *blockDim.x* is the amount of threads in the block and *gridDim.x* is the amount of blocks in the kernel (Harris, 2017a). For the OpenCL implementation, we take advantage of the work-group functions to compute the values as follows, considering a two-dimensional work-group arrangement:

$$i_0 = \text{global_id}_0 * \text{global_size}_1 + \text{global_id}_1$$

$$\text{stride} = \text{workgroup_dimension}_0 * \text{workgroup_dimension}_1$$

In order to assess the parallel performance of our algorithm, we use the observed speedup, which is calculated as follows:

$$\text{Speedup} = \frac{T_s}{T_p}$$

where T_s is the wall clock time of serial execution and T_p is the wall clock time of parallel execution (Barney, 2017).

3.3 Cluster architecture

The message passing interface (MPI) is the standard protocol for memory parallelisation. It provides an efficient parallel programming model for distributed memory systems, which makes it a widely used protocol in a number of applications (Wu et al., 2016; Eberius et al., 2017). Open MPI is an open source implementation of the protocol that has been quite popular in high-performance computing applications. The project has

collaborations from research, academic and industry members, which make it one of the best MPI implementations available (Eberius et al., 2017; Open MPI: Open Source High Performance Computing, 2017).

Open MPI eases the process of creating parallelised applications that run over clusters of computing devices. It also allows performance evaluation of parallel implementations using the profiling interface defined in the protocol (PMPI) (Eberius et al., 2017). Once computations are distributed through the computing devices, libraries such as POSIX, OpenMP, CUDA, or OpenCL can provide further parallelisation alternatives inside each Open MPI process. Moreover, the communication between the nodes uses the secure sockets layer (SSL) protocol, which provides secure information exchange through the network.

The Open MPI implementation establishes a constant number of four instances. It assigns one process per instance, considering the first instance as the master node and the remaining instances as the slave nodes. Inside each process, we use the OpenMP implementation to create the threads for each instance. After each instance runs its threads, the maximum nonce, its header and the block hash value is sent back to the master node. Inside the master node process, results are consolidated and the maximum nonce is then printed with its corresponding block hash.

3.4 Power consumption

As stated before, DLTs have a drawback regarding wastage of computer power and energy (Yli-Huoma et al., 2016; Paul et al., 2014). For instance, the blockchain supporting bitcoin transactions consume an estimated 45.64 TWh of annual electricity. Such amount is close to the electricity consumption of Iraq. Moreover, it represents an annual carbon footprint of 22364 CO₂ kt (Bitcoin Energy Consumption Index, 2018), which make blockchain be considered an environmental disaster (Virtual Bitcoin Mining Is a Real-World Environmental Disaster, 2018). Therefore, it is essential to measure the power consumption of the hash generation phase – the most demanding phase of the whole blockchain protocol.

Fan et al. (2007) considers CPU utilisation as the primary factor in machine power consumption. They propose a linear model to describe power usage. Using the same approach and considering the CPU core utilisation throughout the whole hash generation phase, we compute the power usage as:

$$P_U = \frac{P_{SPEC} U_{CPU} T_P}{0.85} \quad (1)$$

where P_{spec} is the power rating specified in the processing unit datasheet, U_{CPU} is the CPU usage and T_P is the wall clock time of the hash generation execution. We assume a power supply efficiency of 85% (Fan et al., 2007; Rivoire et al., 2008).

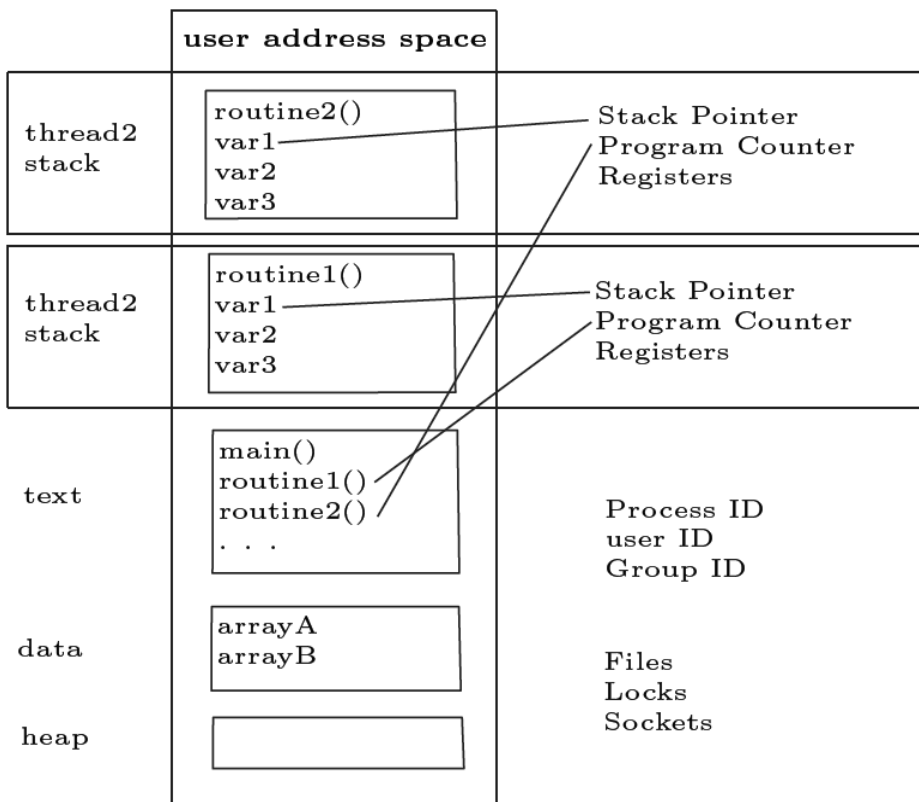
4 Experimental results

The parallelisation strategies were tested using a computer with 8 GB of RAM, an Intel Core i7-7700HQ CPU (Intel Core i7-7700HQ Processor, 2018) and a GPU NVidia GeForce GTX 1050 (GEFORCE GTX 1050, 2018a) with 2 GB of dedicated RAM. The

CPU has four physical cores and eight threads through hyperthreading. Its base frequency is 2.8 GHz, while the Max Turbo frequency is 3.8 GHz. It also has a 6 MB cache with 64-byte cache line length. On the other hand, the GPU is part of the Pascal architecture. It has 640 CUDA cores arranged in five streaming multiprocessors. Its clock runs at 1,354 MHz, with a boost of $1.3 \times$. Its memory interface width is 128-bits. The cluster for Open MPI was set up using the google cloud platform (GCP). It comprises four instances with 4 CPUs and 3.6 GB RAM each (n1-highcpu-4 machine type). The underlying CPU of the virtual instances is an Intel Xeon CPU (Intel Xeon Processor E5345, 2018) at 2.30 GHz.

To get data from a real blockchain application, we used the values from block #453194 and user with public key *1B25f9nd8mDTnkiDf6PED2twh5DfaNGxyR*. All data is publicly available at <https://blockchain.info>. Instead of using the Hash 160 of the user address public key, we encode the address using SHA256 and then form the block starting at nonce = 0. Also, we use the timestamp of the block mentioned above (15 February 2017 17:03:57) for the time field, instead of the current time. The example header, with all fields in hexadecimal format, appears in Table 2. We iterate the nonce from 0 to the valid nonce of block #453194, which has a value of 4003888472.

Figure 2 Unix threads share process resources and exist within the parent process



Source: From Barney (2017)

Table 2 Example header for hash generation evaluation

<i>Version</i>
20000000
<i>Previous hash</i>
000000000000000000002859a549a49c2c7 0b92efeef7d1493c3574fa54b0568f88
<i>Merkle root</i>
ae99c062836fc721f4a2489a6589a402 40a40d77fd70442c0cc1cf417ca729c6
<i>Time</i>
58a4d04d
<i>Address</i>
d8a7bb9a31d15988a10c2648942eb7be b89dc772fdbe5800e7d247ae6042784f
<i>Nonce</i>
0 to 4003888472

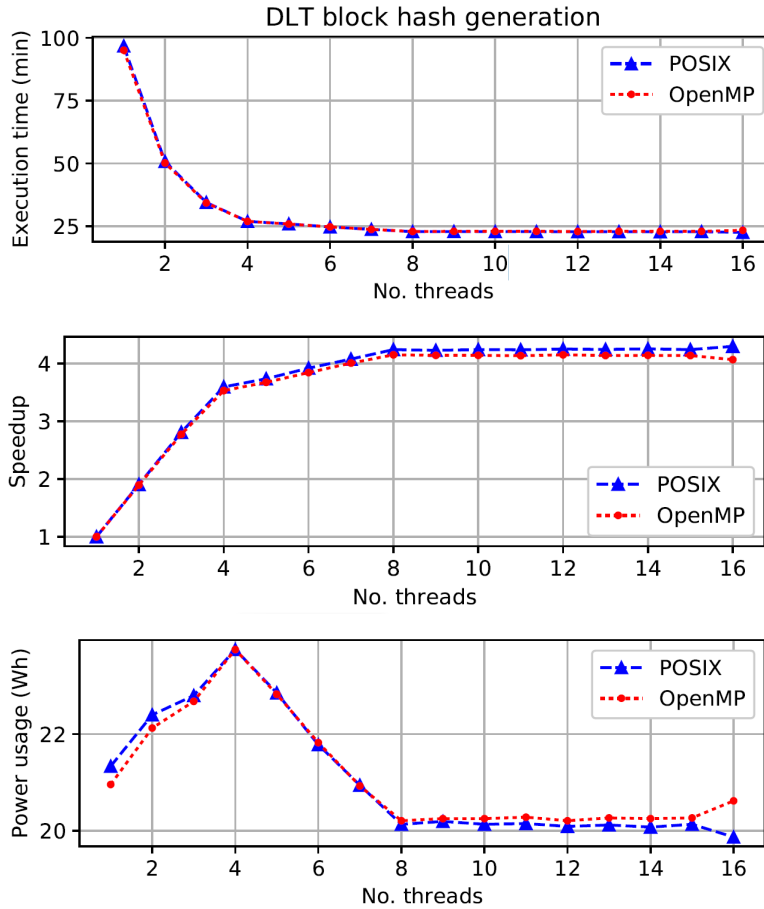
Notes: All values in hexadecimal format.
Source: It uses public information from <https://blockchain.info>

4.1 CPU

Figure 3 depicts the result when running the algorithm using POSIX and OpenMP to parallelise the hash generation. There seems to be no plausible difference in speedup between POSIX and OpenMP implementations. This happens because OpenMP is a library that eases the parallel programming process. But under the hood, programs generated using OpenMP share the exact same threading and memory management mechanisms that POSIX uses for its implementation. Their only difference comes from the software development process and the way OpenMP allows a faster implementation of parallel programs. When we are using OpenMP, we do not need to manually implement the threading mechanisms. Thus, we develop our software solutions faster. Also, the approach is less error-prone, as the parallelising tasks are arranged at compile time.

No improvement in computational efficiency was observed when manually padding the data structure used for storing thread results, which suggests that the compiler automatically optimised the data structure size to match the cache line size of the processor cores. Padding by creating a matrix structure did not improve the computational efficiency mainly because the data structure has a width of 108 bytes, way above cache line size of 64 bytes.

Figure 3 Experiment running OpenMP and POSIX parallelised hash generation algorithm (see online version for colours)

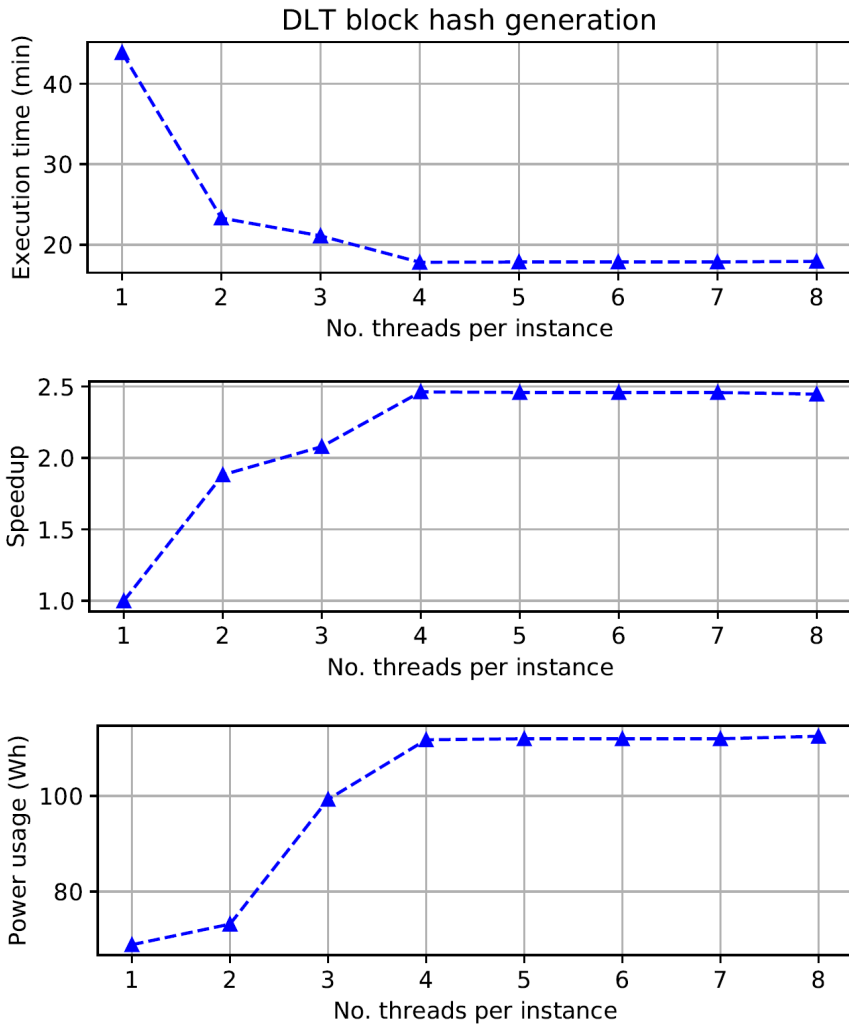


Notes: Results from one thread per instance to eight threads per instance. Speedup is calculated against one thread per instance.

4.2 GPU

A considerable speedup is observed when running the hash generation phase on the GPU with CUDA. The execution time improves from 1,351 seconds – CPU with 16 threads – to 120 seconds with 28 GPU blocks, 128 threads per block (Figures 3 and 5). The speedup was calculated against the fastest CPU time, which was obtained using 16 threads. It is important to note that the GPU enables a $10\times$ increase in computational efficiency (speedup of 11.28 with CUDA and 12.74 with OpenCL). Such increase matches the expected speedup with this type of parallel platform. Different computational intensive applications – for instance, neural network training – also report $10\times$ increases in speed of processing when using GPUs.

Figure 4 Experiment running Open MPI parallelised hash generation algorithm (see online version for colours)



Notes: Results from one thread per instance to eight threads per instance. Speedup is calculated against one thread per instance.

To evaluate performance with the CUDA GPU implementation, we took the time per block, assigning to each block 128 threads (Figure 5), which is the maximum number of threads available for the GPU. No improvement was observed when doubling the threads per block to 256 S. Indeed, some seconds were added to the execution time, increasing it from 122 seconds to 130 seconds when running 24 blocks.

The OpenCL implementation also represents a considerable speedup against the fastest CPU time (Figure 5). Computation time decreases from 1,351 seconds – CPU with 16 threads – to 106 seconds when running 24×128 work-groups. It is faster than the CUDA implementation results with 24 blocks $\times$ 128 threads. However, we observe that as the block / work-group count decreases, the CUDA implementation for this particular

GPU has a better performance. Such behaviour could be explained by the fact that, with CUDA, the manual assignment of blocks and threads is tailored to the GPU streaming multiprocessor (SM) and core counts. On the other hand, OpenCL automatically chooses the distribution of work-groups, which explains its better performance as the work-group count approaches the end of the speedup curve. It also accounts for the changing behaviour of the OpenCL speedup curve – with a linear behaviour and performance spikes when the first dimension of the work-groups is a multiple of eight.

When the number of threads exceeds the number of cores in the CPU, no performance improvement is observed (Figure 3). However, increasing the number of blocks / workgroups over the amount of GPU SMs available does continue to improve the algorithm speedup considerably, as can be seen in Figure 5. Nonetheless, when launching 32 blocks with CUDA, computation time increases to 160 s, indicating a limit in performance.

4.3 Cluster

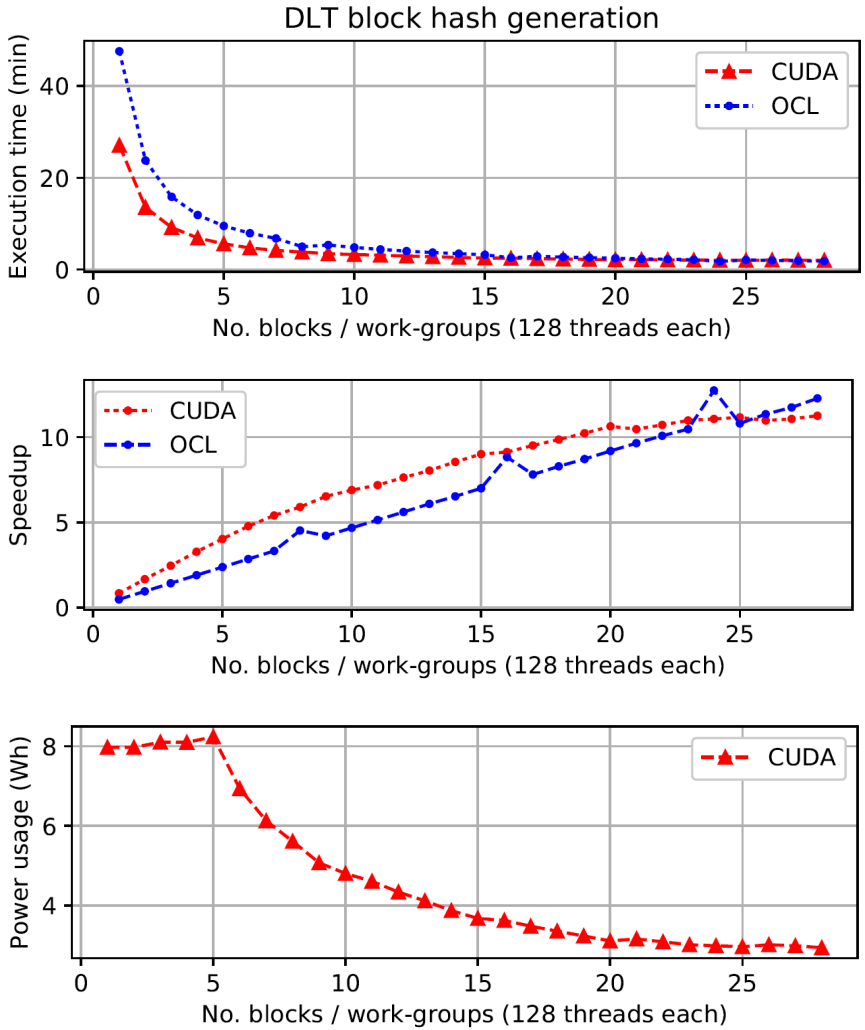
Open MPI experiment results (Figure 4) show an increase in performance when extending from one to four threads per instance. However, additional increases in thread count do not represent any speedup because each instance has only four CPUs. By the same token, the best result is around 18 minutes, which is faster than the best POSIX result because overall we have a total of 16 virtual threads. However, to make a more precise comparison, the execution time with eight virtual threads is 1399s while the execution time with eight POSIX threads is 1,374 s. Thus, Open MPI implementation results are slower than the OpenMP and POSIX experiments. This happens because the processing unit in the Open MPI implementation is virtualised and we have a slower CPU under the virtual instances.

4.4 Power consumption

The hash generation phase takes a 100% utilisation of the processing unit cores during the whole execution. Thus, we use equation 1 to compute power usage for the parallelisation approaches. Power consumption for Intel 7 CPU is 45W (Intel Core i7-7700HQ Processor, 2018), representing 11.25W per core. Intel Xeon 6 CPU has a power consumption of 80W (Intel Xeon Processor E5345, 2018), or 20W per core. On the other hand, power consumption for the NVidia GPUGTX1050 is 75W (GEFORCE GTX 1050, 2018a), representing 15 W per block. We only take data from the CUDA implementation for GPU power usage calculations because we manually control the number of blocks, while the OpenCL implementation automatically controls thread and block counts.

In order to compare power consumption, we calculate power usage using results from Dev's (2014) research and the model from equation (1) (see Table 3). Dev (2014) uses an Intel 7 CPU (Intel Core i7-7700HQ Processor, 2018), which has a power consumption of 45W. The GPU is an NVidia GTX 550 with 116W of power consumption. We use the hash rates reported in Dev (2014) and the maximum nonce in this article (4003888472) to compute the hash generation time required for the power consumption model. The hash rate for the CPU is 1.9M/s, while the hash rate for the GPU is 20.4M/s (GEFORCE GTX 550, 2018b).

Figure 5 Experiment running parallelised hash generation algorithm with CUDA and OpenCL (see online version for colours)



Notes: Results from 1 block to 28 blocks, assigning 128 threads per block with CUDA.
OpenCL implementation ranges from 1×128 work-groups to 28×128 work-groups. Speedup is calculated against the best POSIX result.

Power usage results are depicted in Figures 3 to 5. Mean power usage appears in Table 3. As expected, GPU implementations provide the best energy performance because the hash generation phase is intensive in terms of processing power. Moreover, the POSIX and OpenMP implementations are more energy efficient than the implementation using virtual instances, mainly because of the power efficiency in the Intel Core i7-7700HQ CPU. It is designed for mobile devices – laptops – rather than servers. Also, it pertains to a newer processor generation. Both POSIX and OpenMP results show a better power performance than the results computed from Dev (2014). The CUDA implementation is

more efficient as well. However, it is important to note that the GPU 550 is older than the GPU 1050.

Table 3 Mean power usage in Wh for the different parallelisation approaches

<i>Approach</i>	<i>Power usage (Wh)</i>	<i>Power computations (Wh)</i>
Open MPI	100.20	—
POSIX	21.05	30.99
OpenMP	21.10	30.99
CUDA	4.66	7.44

Source: The table includes power usage computations based on Dev's (2014) research

5 Conclusions

In this work we have explored a number of parallelising techniques, including POSIX, OpenMP, Open MPI, CUDA and OpenCL, to improve computation time in a blockchain hash generation phase. The hash generation phase is part of a democratic blockchain protocol, where computing power is not a deciding factor on the number of blocks that a node can create. Using a GPU with CUDA or OpenCL, we can achieve a timespan of around two minutes, considering a final nonce value similar to a block in a real blockchain application. The GPU allows the algorithm parallelisation with a cyclic approach and generates a 10x speedup in the hash generation phase. We also estimated the amount of power usage for the parallelisation approaches because it is one of the major drawbacks of DLTs. The implementation using GPUs provides the best energy performance.

5.1 Future work

As a collision for the SHA-1 hashing algorithm has already been published (Stevens et al., 2017), existing blockchain architectures using the SHA-1 algorithm for any digest calculation should upgrade to stronger alternatives, such as SHA256 or SHA512. Both SHA256 and SHA512 have larger portions of bits dedicated to security. They are still considered secure hashing algorithms - without published collisions - as the time of this writing.

DLTs have some challenges that need further research. Blockchain research should continue to evaluate technical weaknesses and propose solutions for them. It is necessary to create more applications based on the decentralised technology, apart from the popular cryptocurrencies. Moreover, researchers should assess proposed approaches, evaluating their performance in real implementations. Also, more research about scalability should be conducted. For the blockchains to become ubiquitous, they need to be scalable and efficient (Yli-Huuma et al., 2016).

References

- Angermueller, C., Pärnamaa, T., Parts, L. and Stegle, O. (2016) ‘Deep learning for computational biology’, *Molecular Systems Biology*, Vol. 12, No. 7, p.878.
- Atlidakis, V., Andrus, J., Geambasu, R., Mitropoulos, D. and Nieh, J. (2016) ‘POSIX abstractions in modern operating systems: the old, the new, and the missing’, in *Proceedings of the Eleventh European Conference on Computer Systems*, ACM, p.19.
- Barney, B. (2017) *High Performance Computing* [online] <https://computing.llnl.gov/tutorials/> (accessed 10 December 2017).
- Bitcoin Energy Consumption Index (2018) [online] <https://digiconomist.net/bitcoin-energyconsumption> (accessed 29 January 2018).
- Descriptions of SHA-256, SHA-384, and SHA-512 (2017) [online] <http://www.iwar.org.uk/comsec/resources/cipher/sha256-384-512.pdf> (accessed 10 December 2017).
- Dev, J.A. (2014) ‘Bitcoin mining acceleration and performance quantification’, in 27th *Canadian Conference on Electrical and Computer Engineering (CCECE)*, IEEE, pp.1–6.
- Eberius, D., Patinyasakdikul, T. and Bosilca, G. (2017) ‘Using software-based performance counters to expose low-level open MPI performance information’, in *Proceedings of the 24th European MPI Users’ Group Meeting*, ACM, p.7.
- Fan, X., Weber, W-D., and Barroso, L.A. (2007) ‘Power provisioning for a warehouse-sized computer’, in *ACM SIGARCH Computer Architecture News*, ACM, Vol. 35, No. 2, pp.13–23.
- GEFORCE GTX 1050 (2018a) <https://www.nvidia.com/enus/geforce/products/10series/geforce-gtx-1050/> (accessed 29 January 2018).
- GEFORCE GTX 550 (2018b) [online] <https://www.geforce.com/hardware/desktop-gpus/geforce-gtx-550ti/specifications> (accessed 29 January 2018).
- Harris, M. (2017) *An Even Easier Introduction to CUDA* [online] <https://devblogs.nvidia.com/parallelforall/even-easier-introduction-cuda/> (accessed 10 December 2017).
- Harris, M. (2017) *Unified Memory for CUDA Beginners* [online] <https://devblogs.nvidia.com/parallelforall/unified-memory-cuda-beginners/> (accessed 10 December 2017).
- Intel Core i7-7700HQ Processor (2018) [online] https://ark.intel.com/products/97185/Intel-Core-i7-7700HQ-Processor-6M-Cache-up-to-3_80-GHz (accessed 29 January 2018).
- Intel Xeon Processor E5345 (2018) [online] https://ark.intel.com/products/28032/Intel-Xeon-Processor-E5345-8M-Cache-2_33-GHz-1333-MHz-FSB (accessed 29 January 2018).
- Liebling, S.L. and Khanna, G. (2017) ‘Scalar collapse in ads with an opencl open source code’, *Classical and Quantum Gravity*, Vol. 34, No. 20, p.205012.
- Michail, H.E., Athanasiou, G.S., Theodoridis, G., Gregoriades, A. and Goutis, C.E. (2016) ‘Design and implementation of totally-self checking SHA-1 and SHA-256 hash functions architectures’, *Microprocessors and Microsystems*, Vol. 45, Part B, pp.227–240.
- Nakamoto, S. (2008) *Bitcoin: A Peer-To-Peer Electronic Cash System* [online] <http://www.Bitcoin.org> (accessed 10 December 2017).
- Open MPI: Open Source High Performance Computing (2017) [online] <https://www.open-mpi.org/> (accessed 10 December 2017).
- Pas, R.V.D. (2009) *An Overview of OpenMP, Performance Evaluation for the Hash Generation Phase 17* [online] <http://www.openmp.org/wpcontent/uploads/ntu-vanderpas.pdf> (accessed 10 December 2017).
- Paul, G., Sarkar, P. and Mukherjee, S. (2014) ‘Towards a more democratic mining in bitcoins’, in *International Conference on Information Systems Security*, Springer, pp.185–203.
- Pilkington, M., Crudu, R. and Grant, L.G. (2017) ‘Blockchain and bitcoin as a way to lift a country out of poverty-tourism 2.0 and e-governance in the republic of moldova’, *International Journal of Internet Technology and Secured Transactions*, Vol. 7, No. 2, pp.115–143

- Rivoire, S., Ranganathan, P. and Kozyrakis, C. (2008) 'A comparison of high-level full-system power models', *HotPower*, Vol. 8, No. 2, pp.32–39.
- Sakharnykh, N. (2016) *Beyond GPU Memory Limits with Unified Memory on Pascal* [online] <https://devblogs.nvidia.com/parallelforall/beyond-gpu-memory-limitsunified-memory-pascal/> (accessed 10 December 2017).
- Stevens, M., Bursztein, E., Karpman, P., Albertini, A. and Markov, Y. (2017) 'The first collision for full SHA-1', *IACR Cryptology ePrint Archive*, Vol. 10401, No. 1, pp.570–596.
- Taha, M.A., El Assad, S., Queudet, A. and Deforges, O. (2017) 'Design and efficient implementation of a chaos-based stream cipher', *International Journal of Internet Technology and Secured Transactions*, Vol. 7, No. 2, pp.89–114.
- The Current State of the Healthcare Blockchain (2017) [online] <https://medium.com/@ConnectedDots/the-current-state-of-the-healthcare-blockchain-2d182d04c088> (accessed 10 December 2017).
- The Open Standard for Parallel Programming of Heterogeneous Systems (2017) [online] <https://www.khronos.org/opencv/> (accessed 10 December 2017).
- Virtual Bitcoin Mining Is a Real-World Environmental Disaster (2018) [online] <https://www.bloomberg.com/news/articles/2013-04-12/virtual-bitcoin-mining-is-a-real-world-environmental-disaster> (accessed 29 January 2018).
- Wu, W., Bosilca, G., Vandevaart, R., Jeaugey, S. and Dongarra, J. (2016) 'GPU aware non-contiguous data movement in open MPI', in *Proceedings of the 25th ACM International Symposium on High-Performance Parallel and Distributed Computing*, ACM, pp.231–242.
- Yli-Huoma, J., Ko, D., Choi, S., Park, S. and Smolander, K. (2016) 'Where is current research on blockchain technology? A systematic review', *PloS One*, Vol. 11, No. 10, p.e0163477.

Notes

- 1 <http://www.butterflylabs.com/>.
- 2 <https://github.com/ckolivas/cgminer>.
- 3 <https://github.com/luke-jr/bfgminer>.
- 4 <http://www.openmp.org/>.
- 5 <https://developer.nvidia.com/cuda-toolkit>.
- 6 <https://www.khronos.org/registry/OpenCL/sdk/1.0/docs/man/xhtml/clBuildProgram.html>.